

NEURAL NETWORK AND DEEP LEARNING LAB MANUAL

Ex no:01

Date:

ImplementsimplevectoradditioninTensorFlow.

Aim:

ImplementsimplevectoradditioninTensorFlow.

Algorithm:

1. Input:Twovectorsofthesamelength.
2. Output: A vector obtained by adding corresponding elements of the input vectors.
3. Procedure:
4. Initializeanemptyvector.
5. Iterateovereachelementoftheinputvectors
.
6. Addcorrespondingelementstogetherand appendtheresulttotheoutputvector.
7. Returntheoutputvector.

Program:

```
import tensorflow as tf

# Define two vectors
vector1 = tf.constant([1, 2, 3])
vector2 = tf.constant([4, 5, 6])

# Add the vectors
result = tf.add(vector1, vector2)

# Start a TensorFlow session
with tf.Session() as sess:
    # Run the session to compute the result
    output = sess.run(result)
    print("Result of vector addition:", output)
```

Output:

Result of vector addition: [5 7 9]

Result:

Exno:02

Date:

ImplementaregressionmodelinKeras.

Aim:

ImplementaregressionmodelinKeras.

Algorithm:

1. ****Input****: Training data (X) and (y) , model architecture, training parameters.
2. ****DefineModel****:
 - Create a Sequential model.
 - Add layers with `Dense` specifying neurons and activation.
3. ****CompileModel****:
 - Compile with optimizer, loss function, and metrics.
4. ****TrainModel****:
 - Use `fit` with (X) , (y) , epochs, and batch size.
5. ****Output****:
 - Trained model parameters.
 - Predictions using `predict`.

Program:

```
import numpy as np
from keras.models import Sequential
from keras.layers import Dense

# Generate some random data for demonstration
np.random.seed(0)
X = np.random.rand(100, 1)
y = 2 * X + 1 + 0.1 * np.random.randn(100, 1)

# Define the model
model = Sequential()
model.add(Dense(1, input_shape=(1,))) # Single dense layer
# with one neuron
model.compile(optimizer='sgd',
              loss='mean_squared_error') # Using stochastic gradient descent as optimizer

# Train the model
model.fit(X, y, epochs=100)

# Make predictions
predictions = model.predict(X)

# Print the trained parameters
print("Trained parameters:")
for layer in model.layers:
    print(layer.get_weights())
```

Output:

Epoch1/100

100/100[=====]-0s2ms/step-loss:1.2966

Epoch2/100

100/100[=====]-0s47us/step-loss:0.3617

...

Epoch100/100

100/100[=====]-0s40us/step-loss:0.0117

Trainedparameters:

[array([[1.9914212]],dtype=float32),array([1.0113094],dtype=float32)]

Result:

Exno:03

Date:

Implementaperceptronin TensorFlow/KerasEnvironment.

Aim:

ImplementaperceptroninTensorFlow/Keras
Environment.

Algorithm:

1. ****Input****: Training data (X) and labels (y) , model architecture, training parameters.
2. ****DefineModel****:
 - Create a Sequential model.
 - Add a single dense layer with 1 neuron and sigmoid activation.
3. ****CompileModel****:
 - Compile with optimizer (e.g., SGD), binary cross-entropy loss, and accuracy metric.
4. ****TrainModel****:
 - Use `fit` with (X) , (y) , epochs, and verbose set to 0.
5. ****EvaluateModel****:
 - Evaluate the model's loss and accuracy on the training data using `evaluate`.
6. ****MakePredictions****:
 - Use `predict` to make predictions on the training data.

Program:

```
import numpy as np
from keras.models import Sequential
from keras.layers import Dense

# Define the data
X_train = np.array([[0,0],[0,1],[1,0],[1,1]])
y_train = np.array([0,0,0,1])

# Define the perceptron model
model = Sequential()
model.add(Dense(1, input_dim=2, activation='sigmoid'))

# Compile the model
model.compile(optimizer='sgd',
              loss='binary_crossentropy', metrics=['accuracy'])

# Train the model
model.fit(X_train, y_train, epochs=1000, verbose=0)

# Evaluate the model
loss, accuracy = model.evaluate(X_train, y_train)
print("Loss:", loss)
print("Accuracy:", accuracy)

# Make predictions
predictions = model.predict(X_train)
print("Predictions:", predictions.flatten().round())
```


Output:

```
1/1 [=====]  
Loss: 0.5091173052787781  
Accuracy: 0.75  
1/1 [=====]  
Predictions: [0. 0. 0. 0.]
```

Result:

Exno:04

**Date: ImplementaFeed-ForwardNetworkin
TensorFlow/Keras.**

Aim:

ImplementaFeed-ForwardNetworkin
TensorFlow/Keras.

Algorithm:

1. ****Input****: Training data (X) and labels (y) , model architecture (number of layers, neurons per layer, activation functions), training parameters (optimizer, loss function, number of epochs).
2. ****DefineModel****:
 - Create a Sequential model.
 - Add dense layers with specified neurons and activation functions.
3. ****CompileModel****:
 - Compile with optimizer, loss function, and metrics.
4. ****TrainModel****:
 - Use `fit` with (X) , (y) , epochs, and verbose set to 0.
5. ****EvaluateModel****:
 - Evaluate the model's loss and accuracy on the training data using `evaluate`.
6. ****MakePredictions****:
 - Use `predict` to make predictions on the training data.

Program:

```
import numpy as np
from keras.models import Sequential
from keras.layers import Dense

# Define the data
X_train = np.array([[0,0],[0,1],[1,0],[1,1]])
y_train = np.array([0,1,1,0])

# Define the feed-forward network model
model = Sequential()
model.add(Dense(4, input_dim=2, activation='relu'))
# Hidden layer with 4 neurons and ReLU activation
model.add(Dense(1, activation='sigmoid'))
# Output layer with 1 neuron and sigmoid activation

# Compile the model
model.compile(optimizer='adam',
              loss='binary_crossentropy', metrics=['accuracy'])

# Train the model
model.fit(X_train, y_train, epochs=1000, verbose=0)

# Evaluate the model
loss, accuracy = model.evaluate(X_train, y_train)
print("Loss:", loss)
print("Accuracy:", accuracy)

# Make predictions
predictions = model.predict(X_train)
print("Predictions:", predictions.flatten().round())
```

Output:

```
1/1 [=====]  
Loss: 0.39375635981559753  
Accuracy: 1.0  
1/1 [=====]  
Predictions: [0. 1. 1. 0.]
```

Result:

Exno:05

Date: Implement an Image Classifier using CNN in TensorFlow/Keras.

Aim:

Implement an Image Classifier using CNN in TensorFlow/Keras.

Algorithm:

1. ****Input****: CIFAR-10 dataset.
2. ****PreprocessData****:
 - Load the dataset.
 - Normalize pixel values to the range [0,1].
 - Convert labels to one-hot encoded vectors.
3. ****DefineModel****:
 - Create a Sequential model.
 - Add convolutional layers with ReLU activation and max pooling.
 - Flatten the output.
 - Add dense layers with ReLU activation.
 - Add output layer with softmax activation.
4. ****CompileModel****:
 - Compile with Adam optimizer, categorical cross-entropy loss, and accuracy metric.
5. ****TrainModel****:
 - Use `fit` with training data, epochs, batch size, and validation data.

6. ****EvaluateModel****:

- Evaluate the model's loss and accuracy on the test data.

7. ****Output****:

- Loss and accuracy metrics.

Program:

```
import numpy as np
from keras.datasets import cifar10
from keras.models import Sequential
from keras.layers import Conv2D, MaxPooling2D, Flatten, Dense
from keras.utils import to_categorical

# Load the CIFAR-10 dataset
(X_train, y_train), (X_test, y_test) = cifar10.load_data()

# Preprocess the data
X_train = X_train.astype('float32') / 255.0
X_test = X_test.astype('float32') / 255.0
y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)

# Define the CNN model
model = Sequential()
model.add(Conv2D(32, (3, 3), activation='relu', input_shape=(32,
32, 3))) # Convolutional layer with 32 filters
```

```
model.add(MaxPooling2D((2, 2)))# Max pooling layer
model.add(Conv2D(64, (3, 3), activation='relu'))
#Convolutionallayerwith64filters
model.add(MaxPooling2D((2, 2)))# Max pooling layer
model.add(Conv2D(64, (3, 3), activation='relu'))
#Convolutionallayerwith64filters
model.add(Flatten())#Flattentheoutputfordenselayers
model.add(Dense(64,activation='relu'))#Denselayerwith 64
neurons
model.add(Dense(10,activation='softmax')) #Outputlayer
withsoftmaxactivation

#Compilethemodel
model.compile(optimizer='adam',
loss='categorical_crossentropy',metrics=['accuracy'])

#Trainthemodel
model.fit(X_train, y_train, epochs=10, batch_size=64,
validation_data=(X_test, y_test))

#Evaluatethemodel
loss, accuracy = model.evaluate(X_test, y_test)
print("Loss:",loss)
print("Accuracy:",accuracy)
```

Output:

```
Downloading data from https://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz
170498071/170498071 [=====] - 3s 0us/step
Epoch 1/10
781/782 [=====>.] - ETA: 0s - loss: 1.5744 - accuracy: 0.4277WARNING:tensorflow
782/782 [=====] - 71s 89ms/step - loss: 1.5743 - accuracy: 0.4277 - val_loss:
Epoch 2/10
782/782 [=====] - 66s 85ms/step - loss: 1.1927 - accuracy: 0.5788 - val_loss:
Epoch 3/10
782/782 [=====] - 66s 84ms/step - loss: 1.0462 - accuracy: 0.6321 - val_loss:
Epoch 4/10
782/782 [=====] - 65s 83ms/step - loss: 0.9550 - accuracy: 0.6658 - val_loss:
Epoch 5/10
782/782 [=====] - 64s 82ms/step - loss: 0.8876 - accuracy: 0.6896 - val_loss:
Epoch 6/10
782/782 [=====] - 63s 81ms/step - loss: 0.8263 - accuracy: 0.7115 - val_loss:
Epoch 7/10
782/782 [=====] - 62s 80ms/step - loss: 0.7852 - accuracy: 0.7281 - val_loss:
Epoch 8/10
782/782 [=====] - 62s 79ms/step - loss: 0.7326 - accuracy: 0.7440 - val_loss:
Epoch 9/10
782/782 [=====] - 62s 79ms/step - loss: 0.7015 - accuracy: 0.7539 - val_loss:
Epoch 10/10
782/782 [=====] - 59s 76ms/step - loss: 0.6633 - accuracy: 0.7699 - val_loss:
313/313 [=====] - 5s 15ms/step - loss: 0.8281 - accuracy: 0.7209
Loss: 0.828092098236084
Accuracy: 0.7208999991416931
```

Result:

Exno:06

Date: Improve the Deep learning model by fine tuning hyper parameters.

Aim: Improve the Deep learning model by fine tuning hyper parameters.

Algorithm:

1. Input: Training data, labels, fine-tuned hyperparameters.
2. Define Model:
 - Create a Sequential model.
 - Add layers with specified neurons and activation functions.
3. Compile Model:
 - Compile with fine-tuned optimizer, loss function, and metrics.
4. Train Model:
 - Use fit with training data and fine-tuned epochs and batch size.
5. Evaluate Model:
 - Evaluate loss and accuracy on the training data.
6. Make Predictions:
 - Use predict to make predictions.

Program:

```
import numpy as np
from keras.models import Sequential
from keras.layers import Dense

# Define the data
X_train = np.array([[0,0],[0,1],[1,0],[1,1]])
y_train = np.array([0,0,0,1])

# Define the perceptron model with fine-tuned hyperparameters
model = Sequential()
model.add(Dense(4, input_dim=2, activation='relu'))
# Hidden layer with 4 neurons and ReLU activation
model.add(Dense(1, activation='sigmoid'))
# Output layer with 1 neuron and sigmoid activation

# Compile the model with fine-tuned hyperparameters
model.compile(optimizer='adam', loss='binary_crossentropy',
metrics=['accuracy'])

# Train the model with fine-tuned hyperparameters
model.fit(X_train, y_train, epochs=200, batch_size=4, verbose=0)

# Evaluate the model
loss, accuracy = model.evaluate(X_train, y_train)
print("Loss:", loss)
print("Accuracy:", accuracy)

# Make predictions
predictions = model.predict(X_train)
print("Predictions:", predictions.flatten().round())
```

Output:

```
1/1 [=====] - 0s 271ms/step - loss: 0.6055 - accuracy: 1.0000  
Loss: 0.6054630875587463  
Accuracy: 1.0  
1/1 [=====] - 0s 148ms/step  
Predictions: [0. 0. 0. 1.]
```

Result:

Exno:07

Date: Improve the Deep learning model by fine tuning hyper parameters.

Aim: Improve the Deep learning model by fine tuning hyper parameters.

Algorithm:

1. Load Pre-trained Model: Choose a pre-trained CNN.
2. Remove Top Layers: Discard top layers, keeping the base.
3. Add New Fully Connected Layers: Add dense layers for classification.
4. Freeze Pre-trained Layers: Optionally freeze base weights.
5. Prepare Dataset: Organize and preprocess data.
6. Train the Model: Train on the dataset.
7. Fine-tuning (Optional): Optionally fine-tune parameters.
8. Tune Hyperparameters: Optimize model performance.
9. Evaluate on Test Set: Assess final performance.
10. Deploy the Model: Use for predictions on new data.

Program:

```
import numpy as np
import tensorflow as tf
from tensorflow.keras.applications import VGG16
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.datasets import cifar10
from tensorflow.keras.utils import to_categorical
from sklearn.model_selection import train_test_split

# Load CIFAR-10 dataset
(x_train, y_train), (x_test, y_test) = cifar10.load_data()

# Normalize pixel values to the range [0, 1]
x_train = x_train / 255.0
x_test = x_test / 255.0

# Convert class vectors to binary class matrices
num_classes = 10
y_train = to_categorical(y_train, num_classes)
y_test = to_categorical(y_test, num_classes)

# Split the data into training and validation sets
x_train, x_val, y_train, y_val = train_test_split(x_train, y_train,
test_size=0.2, random_state=42
)
```

```
#Loadpre-trainedVGG16modelwithoutthetoplayers
base_model=VGG16(weights='imagenet',include_top=False,
input_shape=(32,32,3))
```

```
#Freezethepre-trainedlayers
forlayerinbase_model.layers:
    layer.trainable=False
```

```
#Createnewmodelontopofthepre-trainedbasemodel
model=Sequential([
    base_model,
    Flatten(),
    Dense(256,activation='relu'),
    Dense(num_classes,activation='softmax')
])
```

```
#Compilethemodel
model.compile(optimizer=Adam(learning_rate=0.001),loss='categorical_crossentropy', metrics=['accuracy'])
```

```
#Trainthemodel
history=model.fit(
    x_train, y_train,
    batch_size=32,
    epochs=10,
    validation_data=(x_val,y_val))
```

```
#Evaluate the model
loss,accuracy=model.evaluate(x_test,y_test)
print(f'Test Loss: {loss}, Test Accuracy: {accuracy}')

#Save the model
model.save('transfer_learning_model.h5')
```

Output:

```
Epoch1/10
1250/1250[=====]-76s60ms/step-
loss:1.2851-accuracy:0.5554-val_loss:1.0989-val_accuracy: 0.6085
Epoch2/10
1250/1250[=====]-73s59ms/step-
loss: 1.0775 - accuracy: 0.6259 - val_loss: 1.0723 - val_accuracy:
0.6210
Epoch3/10
1250/1250[=====]-74s59ms/step-
loss:1.0159-accuracy:0.6456-val_loss:1.0457-val_accuracy: 0.6293
Epoch4/10
1250/1250[=====]-74s59ms/step-
loss:0.9733-accuracy:0.6594-val_loss:1.0441-val_accuracy: 0.6333
Epoch5/10
1250/1250[=====]-74s59ms/step-
loss:0.9397-accuracy:0.6723-val_loss:1.0389-val_accuracy: 0.6334
Epoch6/10
1250/1250[=====]-74s59ms/step-
loss:0.9093-accuracy:0.6827-val_loss:1.0390-val_accuracy: 0.6341
```

Epoch7/10
1250/1250[=====]-74s59ms/step-
loss:0.8850-accuracy:0.6902-val_loss:1.0444-val_accuracy: 0.6340
Epoch8/10
1250/1250[=====]-74s59ms/step-
loss:0.8622-accuracy:0.6980-val_loss:1.0583-val_accuracy: 0.6315
Epoch9/10
1250/1250[=====]-74s59ms/step-
loss:0.8407-accuracy:0.7052-val_loss:1.0707-val_accuracy: 0.6301
Epoch10/10
1250/1250[=====]-74s59ms/step-
loss:0.8232-accuracy:0.7114-val_loss:1.0677-val_accuracy: 0.6289
313/313[=====]-8s24ms/step-loss:
1.0782-accuracy:0.6288
Test Loss: 1.0782139301300049, Test Accuracy:
0.6287999749183655

Result:

Exno:08

**Date: UsingapretrainedmodelonKerasfor
Transfer Learning.**

Aim: UsingapretrainedmodelonKerasfor
TransferLearning.

Algorithm: